Digital Nurture 5.0 — Situational Assessment

50 Scenario-Based Practice Questions & Detailed Explanations

I. Design Patterns & SOLID (Situational Scenarios)

Q1. You are refactoring a monolithic legacy class `OrderProcessor` that handles validation, payment processing, database persistence, and email notifications. Modifying payment logic recently broke email sending. Which SOLID principle was violated?

A) Single Responsibility Principle (SRP)
B) Liskov Substitution Principle (LSP)
C) Interface Segregation Principle (ISP)
D) Dependency Inversion Principle (DIP)

Q2. An enterprise checkout system needs to swap out payment gateways (PayPal vs. Stripe) dynamically at runtime based on the customer's currency without modifying the core checkout flow code. Which design pattern best solves this problem?

A) Decorator Pattern
B) Strategy Pattern
C) Factory Pattern
D) Adapter Pattern

Q3. A developer creates a subclass `ReadOnlyList` extending `ArrayList`, but overrides `add()` to throw an `UnsupportedOperationException`. Calling code expecting standard `List` behavior breaks at runtime. Which SOLID principle is violated?

A) Open/Closed Principle
B) Single Responsibility Principle
C) Liskov Substitution Principle
D) Interface Segregation Principle

Q4. In a Spring Boot application, you want to automatically log method execution times across all service methods without modifying the source code of any service class. Which structural pattern does Spring AOP utilize to accomplish this?

A) Facade Pattern
B) Proxy Pattern
C) Flyweight Pattern
D) Bridge Pattern

Q5. A developer needs to instantiate a complex `UserReport` object that has 12 optional configuration parameters (e.g., date ranges, filters, formats). Constructor overloading is becoming unmaintainable. Which pattern should be applied?

A) Builder Pattern

B) Prototype Pattern
C) Singleton Pattern
D) Abstract Factory Pattern

Q6. Your microservice receives incoming XML and JSON webhooks from legacy third-party vendors and needs to convert them into a unified internal Event DTO format for processing. Which structural design pattern should be used?

A) Adapter Pattern
B) Composite Pattern
C) Decorator Pattern
D) Proxy Pattern

Q7. A high-level `FinancialService` class directly instantiates concrete database helper classes like `PostgreSQLHelper` using the `new` keyword, making unit testing impossible. Which SOLID principle addresses this coupling?

A) Interface Segregation Principle
B) Open/Closed Principle
C) Dependency Inversion Principle
D) Single Responsibility Principle

Q8. In a React application, multiple child components need to be instantly notified and re-rendered whenever the user's authentication token changes in the central AuthStore. Which behavioral pattern underpins this React Context capability?

A) Mediator Pattern
B) Observer Pattern
C) Command Pattern
D) State Pattern

Q9. A document editor application needs to implement a multi-level Undo/Redo feature by capturing and restoring the internal state of a text document without exposing its private fields. Which design pattern fits this scenario?

A) Memento Pattern
B) Command Pattern
C) Chain of Responsibility
D) Iterator Pattern

Q10. You need to ensure that an expensive database connection pool configuration object is lazily initialized once when the application starts and shared globally across all threads. Which Creational pattern is default in Spring IoC?

A) Prototype Pattern
B) Singleton Pattern
C) Factory Method Pattern
D) Builder Pattern

II. Spring Core, Maven & Hibernate/JPA (Situational Scenarios)

Q11. While analyzing SQL logs in a Spring Boot app, you observe that fetching 50 `Department` records results in executing 51 distinct SQL queries to load their associated `Employee` lists. How should you resolve this performance issue?

A) Change fetch type to `FetchType.EAGER`
B) Use JOIN FETCH in JPQL or `@EntityGraph`
C) Enable Hibernate L2 Cache only
D) Replace `@OneToMany` with `@ManyToMany`

Q12. A user submits an updated profile form. The entity is loaded in Controller A, updated in DTO form, and passed to Service B outside the active Hibernate session. Calling `session.save()` throws an exception or creates duplicates. Which method should be used to re-attach the entity?

A) session.persist()
B) session.update() or session.merge()
C) session.refresh()
D) session.flush()

Q13. Your team adds JUnit 5 and Mockito to `pom.xml`. However, after running `mvn package`, you discover that test libraries are packaged inside the production JAR file. What Maven tag setting corrects this issue?

A) <scope>provided</scope>
B) <scope>test</scope>
C) <scope>runtime</scope>
D) <optional>true</optional>

Q14. You have two beans implementing `NotificationService`: `EmailNotificationService` and `SmsNotificationService`. Autowiring `NotificationService` fails with `NoUniqueBeanDefinitionException`. What is the best way to explicitly specify the primary default bean?

A) Mark one bean with `@Primary` or use `@Qualifier`
B) Rename both service classes
C) Remove `@Service` from one bean
D) Use `@Scope("prototype")`

Q15. A multi-module Maven build fails because Module B attempts to compile against Module A, but Module A's Java files haven't been converted to `.class` files yet. Which standard Maven lifecycle phase generates `.class` files?

A) validate
B) compile
C) process-resources
D) package

Q16. A high-traffic Spring Boot service experiences thread locking and database connection timeouts during peak sales hours. To optimize database connectivity performance, Spring Boot 2+/3+ relies on which connection pool by default?

A) Apache Commons DBCP2
B) HikariCP
C) Tomcat JDBC Pool
D) C3P0

Q17. You define a `@OneToMany` relationship between `Order` and `OrderItem`. Without additional annotations, JPA creates an unwanted join table `order_order_item`. How do you map this cleanly without a join table?

A) Add `@JoinColumn` on the child side and `mappedBy` on the parent side
B) Use `@Embedded` on `OrderItem`
C) Annotate both classes with `@Table`
D) Change relationship to `@ManyToOne` only

Q18. An application repeatedly executes `SELECT` queries for static reference data (e.g., Country Codes). You want Hibernate to cache these query results across multiple user sessions to eliminate DB queries. What must be enabled?

A) First-level Session cache
B) Second-level cache (L2) and Query Cache
C) Spring Batch processing
D) Hibernate dirty checking

Q19. A legacy database table uses a composite primary key consisting of `company_id` and `employee_id`. What JPA annotation approach allows mapping this composite key via a dedicated key class?

A) @IdClass or @EmbeddedId
B) @CompositeKey
C) @PrimaryKeyJoinColumn
D) @ForeignKey

Q20. A developer calls `session.persist(newCustomer)` inside an active transaction. They notice that no SQL `INSERT` statement is logged immediately, unlike `session.save()`. Why does `persist()` behave this way?

A) persist() requires manual commit calls
B) persist() defers SQL INSERT execution until flush/commit time and returns void
C) persist() only works on cached entities
D) persist() is asynchronous by default

III. Spring Boot & REST APIs (Situational Scenarios)

Q21. DevOps requires an automated system endpoint to monitor whether your Spring Boot application's database connection, disk usage, and services are operational. Which dependency provides `/actuator/health`?

A) spring-boot-starter-web
B) spring-boot-starter-actuator

C) spring-boot-starter-security

D) spring-boot-admin-client

Q22. A REST endpoint returning a `Customer` object renders an HTML template error in the browser instead of returning JSON. What annotation should replace `@Controller` on the class definition?

A) @RestController

B) @ResponseBodyController

C) @JsonController

D) @Service

Q23. A client app wants to update only the `phoneNumber` field of a `User` entity without resending or overwriting the rest of the user payload fields (`firstName`, `email`, etc.). Which HTTP method should be used?

A) PUT

B) POST

C) PATCH

D) UPDATE

Q24. An authenticated user with role `ROLE_USER` attempts to access `/api/v1/admin/delete-user`. The server verifies their identity but rejects access due to insufficient privileges. What HTTP status code must be returned?

A) 401 Unauthorized

B) 403 Forbidden

C) 405 Method Not Allowed

D) 400 Bad Request

Q25. You are designing a RESTful API to retrieve a specific invoice via URL path `/invoices/99812`. Which Spring annotation binds `99812` from the URI path to a method parameter?

A) @RequestParam

B) @PathVariable

C) @RequestHeader

D) @ModelAttribute

Q26. Your microservice needs to invoke an external payment Gateway REST API in a non-blocking, asynchronous manner within a WebFlux reactive application. What HTTP client should replace the legacy blocking `RestTemplate`?

A) HttpClient

B) WebClient

C) AsyncRestTemplate

D) HttpURLConnection

Q27. To maintain clean code across 20 REST controllers, you want to handle `EntityNotFoundException` globally and return a standardized JSON error response. Which annotation should be placed on the global handler class?

A) @ControllerAdvice or @RestControllerAdvice

B) @ExceptionHandler

C) @GlobalResponse

D) @Aspect

Q28. During local testing, your Spring Boot app connects to an H2 database, but in Staging it must connect to PostgreSQL. How should you structure properties files and select the active profile?

A) Create `application-dev.properties` and `application-stage.properties` and set `spring.profiles.active`
B) Use multiple `pom.xml` files
C) Hardcode credentials in Java source code
D) Recompile the JAR for each environment

Q29. A developer wants Spring Boot to automatically configure embedded Tomcat, Jackson JSON serializers, and Spring MVC handlers based on dependencies found in `pom.xml`. Which core annotation enables this?

A) @ComponentScan
B) @EnableAutoConfiguration
C) @Configuration
D) @Service

Q30. A web frontend needs paginated user data (`/users?page=2&size=10&sort=name`). Which Spring Data JPA interface should your repository extend to gain built-in pagination support without writing custom queries?

A) CrudRepository
B) PagingAndSortingRepository
C) JpaSpecificationExecutor
D) QueryByExampleExecutor

IV. React Framework & State Management (Situational Scenarios)

Q31. A developer modifies component state directly using `this.state.count = 5` or `state.user = 'Bob'`. The browser UI fails to re-render. Why does React fail to update the view?

A) Direct mutation does not trigger React's reconciliation process and re-render cycle
B) Direct mutation causes a JavaScript syntax error
C) State can only be modified in class constructors
D) The Virtual DOM deletes mutated variables

Q32. In a complex dashboard, passing a `theme` prop down 7 levels of nested child components creates tedious prop-drilling. What React API or hook allows child components to consume `theme` directly?

A) useReducer
B) useContext / React Context API
C) useCallback
D) useRef

Q33. You render a dynamic list of 500 items using `items.map(...)`. The React console logs a warning, and reordering items causes unexpected UI glitching. What missing prop causes this issue?

A) id prop
B) key prop

C) index prop

D) ref prop

Q34. A functional component needs to execute an API fetch call immediately when the component mounts on screen, but should run ONLY ONCE. How should `useEffect` be written?

A) useEffect(() => { fetch(); }, []) with an empty dependency array

B) useEffect(() => { fetch(); }) without a dependency array

C) useEffect(() => { fetch(); }, [fetch])

D) useEffect(() => { fetch(); }, null)

Q35. A child component `<ExpensiveChart />` re-renders every time its parent component's counter increments, even though `<ExpensiveChart />`'s props haven't changed. How can you prevent unneeded re-renders?

A) Wrap the component with `React.memo()`

B) Use `useRef()` inside the child

C) Replace props with global variables

D) Use `useEffect()` without dependencies

Q36. You need to store a reference to a DOM `<input>` element so that clicking a 'Clear' button automatically calls `.focus()` on the input without causing a component re-render. Which hook is appropriate?

A) useState

B) useRef

C) useMemo

D) useLayoutEffect

Q37. In React Router v6+, you need to map the URL path `/products` to the `<ProductList />` component. Which component syntax inside `<Routes>` is correct?

A) <Route path="/products" element={<ProductList />} />

B) <Route url="/products" component={ProductList} />

C) <Link to="/products" render={<ProductList />} />

D) <Switch path="/products" />

Q38. In a Redux application, when an action `ADD_TO_CART` is dispatched, which pure function calculates the next global state based on the previous state and action?

A) Middleware

B) Reducer

C) Action Creator

D) Store Enhancer

Q39. A component performs heavy array filtering on every re-render whenever an unrelated timer ticks. Which hook can cache the calculated array result until the search query changes?

A) useCallback

B) useMemo

C) useRef

D) useState

Q40. React uses a Virtual DOM memory representation to compare tree structures before writing updates to the browser's actual DOM. What primary benefit does this reconciliation process provide?

A) It eliminates the need for CSS

B) It improves performance by minimizing slow direct manipulations of the real DOM

C) It enforces strict type safety

D) It replaces backend databases

V. Git & Testing with JUnit 5/Mockito (Situational Scenarios)

Q41. You are halfway through implementing a feature on `feature-branch` when an urgent production hotfix requires you to switch immediately to `main`. You don't want to make an incomplete commit. Which Git command temporarily saves your uncommitted changes?

A) git stash

B) git pause

C) git commit --temp

D) git checkout -b

Q42. After committing locally with message `fix: typo in sql query`, you realize you want to clarify the commit message before pushing. Which command allows modifying the most recent commit message?

A) git commit --amend

B) git rebase --edit

C) git commit --update

D) git reset --hard

Q43. Developer A and Developer B edit line 25 of `UserService.java` on different branches. When merging Developer B's branch into `main`, Git pauses and marks the file. What is this scenario called?

A) Fast-forward merge

B) Merge Conflict

C) Divergent Branch Error

D) Detached HEAD State

Q44. When setting up a JUnit 5 test suite, you need to start an in-memory database server ONCE before any unit test methods run in the test class. Which annotation should be placed on the setup method?

A) @BeforeEach

B) @BeforeAll

C) @BeforeClass

D) @SetUp

Q45. You are writing a unit test for `OrderService`. To test `OrderService` in complete isolation without connecting to a real database, you want Mockito to create a fake `OrderRepository`. Which annotation creates this mock object?

A) @Mock

B) @InjectMocks
C) @Spy
D) @Fake

Q46. You have created `@Mock OrderRepository` and `@Mock PaymentGateway`. Which Mockito annotation instantiates the real `OrderService` and automatically injects these mocks into it?

A) @Mock
B) @InjectMocks
C) @Autowired
D) @RunWith

Q47. During a unit test, you want `userRepository.findById(1L)` to return a pre-configured `Optional<User>` instance whenever it is invoked. Which Mockito syntax specifies this behavior?

A) when(userRepository.findById(1L)).thenReturn(Optional.of(testUser));
B) doReturn(testUser).when(userRepository.findById(1L));
C) assert(userRepository.findById(1L)).is(testUser);
D) verify(userRepository).findById(1L);

Q48. You are writing a unit test to verify that calling `userService.getUserById(-1L)` throws `UserNotFoundException`. Which JUnit 5 assertion method verifies exception throwing?

A) assertThrows(UserNotFoundException.class, () -> userService.getUserById(-1L));
B) assertException(UserNotFoundException.class);
C) expectThrows(UserNotFoundException.class);
D) @Test(expected = UserNotFoundException.class)

Q49. In Mockito, if a mocked method returning a non-primitive object (e.g., `userRepository.findByName("John")`) is called without prior `when(...).thenReturn(...)` stubbing, what value is returned by default?

A) An empty new object
B) null
C) NullPointerException
D) Runtime Exception

Q50. A new developer joins your team and needs to copy an existing remote GitHub repository to their local workstation. Which Git command should they run?

A) git copy <url>
B) git clone <url>
C) git checkout <url>
D) git init <url>

Answer Key & Detailed Situational Explanations

Q1: Option A — Single Responsibility Principle (SRP) states a class should have only one reason to change. Monolithic classes handling multiple domain tasks break when one task changes.

Q2: Option B — Strategy Pattern defines a family of algorithms (payment gateways), encapsulates each one, and makes them interchangeable at runtime.

Q3: Option C — Liskov Substitution Principle (LSP) requires subclasses to be substitutable for their base types without breaking client expectation. Throwing unexpected exceptions on base methods violates LSP.

Q4: Option B — Spring AOP uses the Proxy Pattern (JDK Dynamic Proxies or CGLIB) to intercept method calls and apply cross-cutting concerns like logging or security.

Q5: Option A — Builder Pattern isolates complex object construction from its representation, providing a clear fluent API when dealing with numerous optional parameters.

Q6: Option A — Adapter Pattern converts the interface of a class/payload into another interface that clients expect, bridging incompatible DTO formats.

Q7: Option C — Dependency Inversion Principle (DIP) states high-level modules should depend on abstractions (interfaces) rather than concrete implementations (`new PostgreSQLHelper()`).

Q8: Option B — Observer Pattern defines a 1-to-N dependency where state updates in a Subject (Context/Store) notify all registered Observers (React components).

Q9: Option A — Memento Pattern captures and externalizes an object's internal state without violating encapsulation, enabling multi-level undo/redo functionality.

Q10: Option B — Spring IoC containers manage beans as Singletons by default, ensuring a single shared instance exists per application context.

Q11: Option B — N+1 select problem is resolved using `JOIN FETCH` in JPQL or `@EntityGraph`, which fetches parent and children in a single SQL query.

Q12: Option B — `session.update()` or `session.merge()` re-attaches detached objects to the current persistence context, preventing duplicate insertion errors.

Q13: Option B — Setting `<scope>test</scope>` in `pom.xml` ensures test dependencies are available during compilation/testing but excluded from the production build artifact.

Q14: Option A — `@Primary` marks a preferred default bean when multiple candidates exist, while `@Qualifier("beanName")` explicitly selects a specific bean at the injection point.

Q15: Option B — The `compile` phase in the Maven default lifecycle compiles project source code into bytecode (`.class` files).

Q16: Option B — HikariCP is default in Spring Boot 2+ due to its superior light-weight performance, low latency, and efficient connection pooling.

Q17: Option A — Adding `@JoinColumn` on the child entity (`OrderItem`) and `mappedBy` on the parent (`Order`) maps a clean foreign-key relationship without generating a join table.

Q18: Option B — Hibernate L2 Cache (shared across sessions) combined with Query Cache stores SQL result sets in memory, avoiding redundant database calls for static data.

Q19: Option A — `@IdClass` or `@EmbeddedId` are standard JPA annotations used to define composite primary keys via a dedicated key class.

Q20: Option B — `persist()` makes an instance managed without forcing an immediate SQL INSERT, deferring DB execution until flush/commit time and returning void.

Q21: Option B — `spring-boot-starter-actuator` exposes production-ready endpoints including `/actuator/health` and `/actuator/metrics`.

Q22: Option A — `@RestController` combines `@Controller` and `@ResponseBody`, instructing Spring to automatically serialize return objects directly into JSON.

Q23: Option C — HTTP PATCH is designed for partial resource modifications, whereas PUT implies replacing the entire resource payload.

Q24: Option B — HTTP 403 Forbidden indicates the server authenticated the caller but refuses access due to insufficient privileges.

Q25: Option B — `@PathVariable` binds template variables extracted directly from the URI path (e.g., `/invoices/{id}`).

Q26: Option B — `WebClient` (part of Spring WebFlux) is the modern, non-blocking, reactive HTTP client replacing the legacy blocking `RestTemplate`.

Q27: Option A — `@ControllerAdvice` / `@RestControllerAdvice` consolidates exception handling globally across controllers using `@ExceptionHandler` methods.

Q28: Option A — Spring Profiles (`application-{profile}.properties`) allow environment-specific configurations activated via `spring.profiles.active`.

Q29: Option B — `@EnableAutoConfiguration` tells Spring Boot to automatically configure beans based on dependencies present on the classpath.

Q30: Option B — `PagingAndSortingRepository` extends `CrudRepository` to add built-in methods for paginated and sorted data retrieval.

Q31: Option A — React relies on immutable state updates via `setState` or `useState` hook setters to trigger component reconciliation and UI re-rendering.

Q32: Option B — `useContext` / React Context API allows deeply nested child components to directly consume context values without prop-drilling through intermediate parents.

Q33: Option B — The `key` prop gives React list elements a persistent identity across re-renders, enabling efficient DOM diffing and reordering.

Q34: Option A — `useEffect(fn, [])` with an empty dependency array executes the side-effect effect function exactly once when the component mounts.

Q35: Option A — `React.memo()` is a higher-order component that memoizes functional components, skipping re-renders if input props remain unchanged.

Q36: Option B — `useRef` creates a persistent, mutable reference object (`ref.current`) that persists across renders without triggering a re-render when mutated.

Q37: Option A — In React Router v6+, routes are defined using `<Route path="/products" element={<ProductList />} />` inside `<Routes>`.

Q38: Option B — Reducers are pure functions taking `(previousState, action)` as parameters and returning the brand-new application state object.

Q39: Option B — `useMemo` memoizes expensive calculated values between renders, recomputing only when specified dependencies change.

Q40: Option B — React's Virtual DOM minimizes expensive browser real-DOM operations by calculating lightweight diffs in memory and applying batched updates.

Q41: Option A — `git stash` temporarily stashes uncommitted changes into a working stack, allowing you to switch branches with a clean working directory.

Q42: Option A — `git commit --amend` updates the message and content of the most recent local commit.

Q43: Option B — A Merge Conflict occurs when Git cannot automatically reconcile conflicting code changes made to the same file lines across merging branches.

Q44: Option B — `@BeforeAll` in JUnit 5 executes once before all test methods in a class (must be annotated on a static method).

Q45: Option A — `@Mock` creates a mock framework instance of a dependency in Mockito.

Q46: Option B — `@InjectMocks` instantiates the class under test and injects declared `@Mock` and `@Spy` fields into it automatically.

Q47: Option A — `when(mock.method()).thenReturn(value)` is standard Mockito syntax for stubbing return values on mock objects.

Q48: Option A — `assertThrows(ExpectedException.class, executable)` is the JUnit 5 standard method for verifying thrown exceptions.

Q49: Option B — Unstubbed Mockito methods returning non-primitive objects return `null` by default.

Q50: Option B — `git clone <url>` downloads an existing remote Git repository and sets up local tracking branches.